

Day 41



Inheritance & extends Keyword in JavaScript:

What is Inheritance?

Inheritance means one class can take (inherit) properties and methods from another class. It helps avoid writing the same code again.

In simple words: A child class can use everything from a parent class.

What is extends keyword?

The extends keyword is used to create a child class from a parent class.

Example:

```
class Child extends Parent {}
```

This means: Child class will automatically have all methods and properties of Parent class (unless overridden).

Example: extends keyword is used to extends the properties of Animal to the Monkey.

```
JS main.js > ...
1  class Animal{
2      constructor(name, color){
3          this.name = name;
4          this.color = color;
5      }
6      run(){
7          console.log(this.name + " is running");
8      }
9      shout(){
10         console.log(this.name + " is shouting");
11     }
12 }
13
14 class Monkey extends Animal{
15     eatBanana(){
16         console.log(this.name + " is eating banana");
17     }
18 }
19
20 let ani = new Animal("Tom","yellow");
21 let mon = new Monkey("Jack","brown");
22
23 ani.shout();
24 mon.eatBanana();
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS E:\JavaScript\Day41-50\Day41> node main.js
Tom is shouting
Jack is eating banana
```

This code defines a base class `Animal` with properties (`name`, `color`) and methods (`run`, `shout`). The `Monkey` class extends `Animal`, meaning it inherits all its properties and methods while adding its own method `eatBanana`. Two objects are created: `ani` (an `Animal`) and `mon` (a `Monkey`). Calling `ani.shout()` prints that Tom is shouting, and `mon.eatBanana()` prints that Jack is eating a banana.

Example: another basic example of inheritance

```

27 // Base class
28 class Vehicle {
29   constructor(brand) {
30     this.brand = brand;
31   }
32   start() {
33     console.log(this.brand + " is starting");
34   }
35 }
36 // Child class that inherits from Vehicle
37 class Car extends Vehicle {
38   honk() {
39     console.log(this.brand + " is honking");
40   }
41 }
42 // Creating objects
43 let v = new Vehicle("Generic Vehicle");
44 let c = new Car("Toyota");
45 // Calling methods
46 v.start(); // Output: Generic Vehicle is starting
47 c.start(); // Output: Toyota is starting (inherited method)
48 c.honk();  // Output: Toyota is honking

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

PS E:\JavaScript\Day41-50\Day41> node main.js
Generic Vehicle is starting
Toyota is starting
Toyota is honking

```

This example shows how a child class can inherit features from a parent class. The `Vehicle` class has a `brand` property and a `start()` method. The `Car` class extends `Vehicle`, so it automatically gets these features and also adds its own `honk()` method. When we create objects from these classes, the `Vehicle` object can only start, while the `Car` object can both start (inherited) and honk (its own method).

Example: another example of inheritance.

```

51 class Device {
52   turnOn() {
53     console.log("Device is turning on.");
54   }
55   turnOff() {
56     console.log("Device is turning off.");
57   }
58 }
59 // Child class
60 class Phone extends Device {
61   makeCall() {
62     console.log("Phone is making a call.");
63   }
64   // Method overriding
65   turnOn() {
66     console.log("Phone is turning on with animation.");
67   }
68 }
69 // Grandchild class
70 class Smartphone extends Phone {
71   openApp(appName) {
72     console.log("Opening app: " + appName);
73   }
74 }
75 let d = new Device();
76 let p = new Phone();
77 let s = new Smartphone();
78 // Calling methods
79 d.turnOn();
80 p.turnOn(); // Overridden method
81 p.makeCall();
82 s.turnOn(); // Inherits overridden version from Phone
83 s.openApp("YouTube");

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

PS E:\JavaScript\Day41-50\Day41> node main.js
Device is turning on.
Phone is turning on with animation.
Phone is making a call.
Phone is turning on with animation.
Opening app: YouTube

```

This example has three levels of inheritance:

Device → Phone → Smartphone.

The `Phone` class overrides the `turnOn()` method from `Device`, and the `Smartphone` class inherits that overridden behavior. Each class adds its own abilities, showing a layered, more advanced structure—all without using `super()`.

Method overriding and Super keyword in JavaScript:

What is Method Overriding?

Method overriding happens when a child class writes its own version of a method that already exists in the parent class.

In simple words: Child class replaces the parent's method with a new one.

Example idea: Parent has a speak() method → Child creates its own speak().

Why do we use Method Overriding?

- To change or customize the behavior of a parent method.
- To add extra features while keeping the parent functionality.

Example: a basic example of overriding



```
1 class Animal {
2   speak() {
3     console.log("Animal makes a sound");
4   }
5 }
6 class Dog extends Animal {
7   speak() { // overriding
8     console.log("Dog barks");
9   }
10 }
11 let d = new Dog();
12 d.speak(); // Dog barks
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS E:\JavaScript\Day41-50\Day41> node .\overriding.js

Dog barks

In this example, the Animal class has a method speak() that prints a generic sound. The Dog class extends Animal and overrides the speak() method by providing its own version that prints "Dog barks." When we create an object d from the Dog class and call d.speak(), the overridden method in Dog is executed instead of the one in Animal, demonstrating how method overriding gives child classes more specific behavior.

What is super Keyword?

super is used to call the parent class's constructor or methods.

Two places where super works:

1. Inside constructor → calls parent constructor

super(args)

2. Inside child method → calls parent method

super.parentMethodName()

Example: a basic example of super keyword.

```
14
15 class Animal {
16     constructor(name) {
17         this.name = name;
18     }
19 }
20 class Dog extends Animal {
21     constructor(name, breed) {
22         super(name); // call Animal constructor
23         this.breed = breed;
24     }
25 }
26 let d = new Dog("Bruno", "Labrador");
27 console.log(d.name, d.breed);
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS E:\JavaScript\Day41-50\Day41> node .\overriding.js
Bruno Labrador

This code shows how the super keyword is used to call a parent class constructor in JavaScript. The Animal class has a constructor that sets the name property. The Dog class extends Animal and adds a new property, breed. Inside the Dog constructor, super(name) calls the Animal constructor so the name property is properly assigned before adding breed. When a new Dog object is created with "Bruno" and "Labrador", it stores both values and prints them as output.

Example: overriding + using super in a method

```
28
29 class Animal {
30     constructor(name, age) {
31         this.name = name;
32         this.age = age;
33     }
34     info() {
35         console.log(`${this.name} is ${this.age} years old.`);
36     }
37 }
38 class Dog extends Animal {
39     constructor(name, age, breed) {
40         super(name, age); // Call parent constructor
41         this.breed = breed;
42     }
43     info() { // overriding the parent method
44         super.info(); // use the parent's version
45         console.log(`${this.name} is a ${this.breed}.`);
46     }
47     bark() {
48         console.log(`${this.name} is barking!`);
49     }
50 }
51 let d = new Dog("Bruno", 3, "Labrador");
52 d.info();
53 d.bark();
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS E:\JavaScript\Day41-50\Day41> node .\overriding.js
Bruno is 3 years old.
Bruno is a Labrador.
Bruno is barking!

In this example, the Animal class defines common properties like name and age along with an info() method to display them. The Dog class extends Animal, adding a new property breed and overriding the info() method to include breed information. Inside the overridden method, super.info() is used to call the parent class's version before adding the dog-specific message. The class also includes a bark() method that belongs only to dogs. When a Dog object named Bruno is created, calling d.info() prints both the animal and dog details, and d.bark() shows its barking behavior.

--The End--